

Documentación del Proyecto Compilador

Etapas: Analizador Semántico (Tabla de Símbolos y AST)

1 División del Trabajo

El desarrollo de la primera fase del Analizador Semántico se abordó de manera colaborativa. Las responsabilidades se distribuyeron de la siguiente manera para la entrega del AST y la Tabla de Símbolos (TS):

- **Diseño e implementación de la Tabla de Símbolos:** Gribaudo Re Francisco y Francisco Andreani.
- **Diseño y construcción de nodos del Abstract Syntax Tree (AST):** Valentín Pastre Thuer.
- **Integración/elaboración de tests y unidades (Ceedling):** Valentín Pastre Thuer, Gribaudo Re Francisco.

2 Decisiones de Diseño, Suposiciones y Aclaraciones

Dado que en esta etapa solo se solicita la Tabla de Símbolos y el AST, el enfoque estuvo en la recolección estructurada de datos, postergando algunas validaciones de tipos para la etapa final.

- **Modularización estricta:** Se asumió que toda la lógica referida a los símbolos debía mantenerse fuera de las reglas del parser. Por esto, los mayores cambios se centralizaron en `symbols-tab.c`, `syntax-tree.c` y sus cabeceras, exportando únicamente las funciones necesarias hacia `parser.y`.

3 Diseño y Decisiones Clave

- **Tabla de Símbolos Doblemente Enlazada:** Tras evaluar distintas alternativas (como arreglos estáticos o tablas hash simples), decidimos implementar la TS como una lista doblemente enlazada personalizada. Justificamos esta decisión porque facilita enormemente el manejo de ámbitos (*scopes*). Cuando se entra o sale de un bloque de código, la bidireccionalidad permite retraer (hacer *pop*) las variables locales de manera eficiente sin necesidad de reconstruir la tabla entera ni arrastrar identificadores obsoletos.

4 Problemas Conocidos y Testing

La elaboración y corrida de los casos de prueba sigue apoyándose en la infraestructura de Ceedling configurada en la etapa anterior.

- **Fugas de Memoria (Memory Leaks) en el Árbol:** Al comenzar el testeo del nuevo código semántico, detectamos que la terminación del compilador dejaba bloques de memoria huérfanos. Se determinó que los nodos hoja del AST (específicamente las cadenas almacenadas para los ID) no se estaban liberando. *Solución:* Se debió implementar un algoritmo de recorrido en post-orden (*post-order traversal*) que garantice liberar primero los hijos y sus valores antes de liberar el nodo padre.
- **Actividades de mitigación:** Para solucionarlo, nos encontramos elaborando pruebas unitarias directas sobre el componente de la lista doblemente enlazada, simulando inserciones manuales sin pasar por el parser. Una vez que la TS maneje listas de hermanos correctamente en entornos aislados, actualizaremos las producciones en `parser.y`